



FOOTBALL LEAGUE DATABASE IMPLEMENTATION

Technical Documentation



Updates	1
Implementation	3
Data Dictionary	4
PostgreSQL Entity-Relationship Diagram	9
Script	10
Testing.....	13

CALLUM FISHER

UPDATES

Excel Tables:

- **Corrected some information** such as the date for a match that was incorrect.
- **Further decomposed the Match table and the Match event tables.** This was done by separating the columns: date, time and week into another table called Schedule with its own primary key Schedule_ID. This is then used as a foreign key in the Match table. I also separated out the friendly column and made a table called Competition. This table is then used to differentiate between friendlies and tournaments. The primary key Competition_ID is then used as a foreign key in the Match table. I also separated the column event type in the Match Event table into its own table called Event Type with its own primary key called Event_Type_ID. This is then used as a foreign key in the Match Event table.
- **Changed the name of the date, time and week columns** to Match_Date, Match_Time and Match_Week to make it easier to differentiate between column names and formats when working in PostgreSQL.
- **Changed the name of the Home_Team and Away_Team** to Home_Team_ID and Away_Team_ID to make it easier to recognise that they are foreign keys.
- **Added a column in Match table called Result** which tells us which team won the game or if it was a draw.

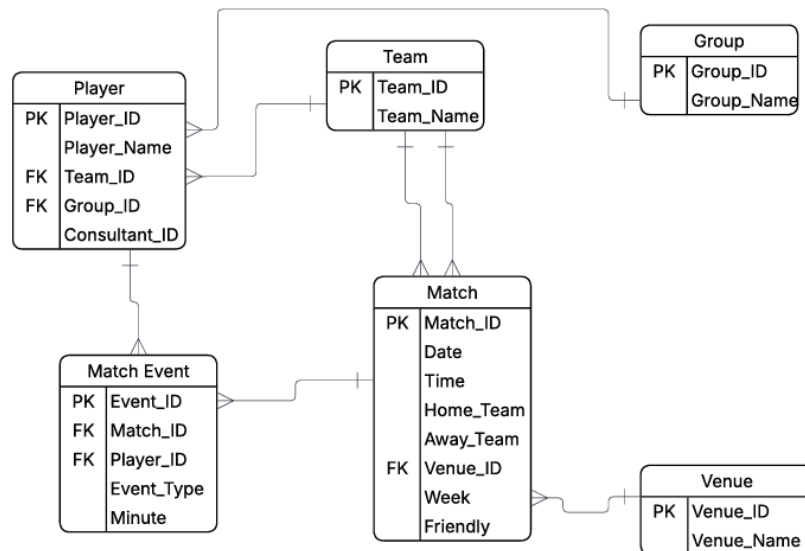
Entity-Relationship Diagram:

- **Added the new entities and fields.**
- **Changed some of the relationships** such as “One player has zero or many match events” and “One match has zero or many match events”.
- **Added constraints and domains.**
- **Added text** to clearly show the relationship between the entities. **The crows feet are also now correlated** with the fields they are showing relationships with.

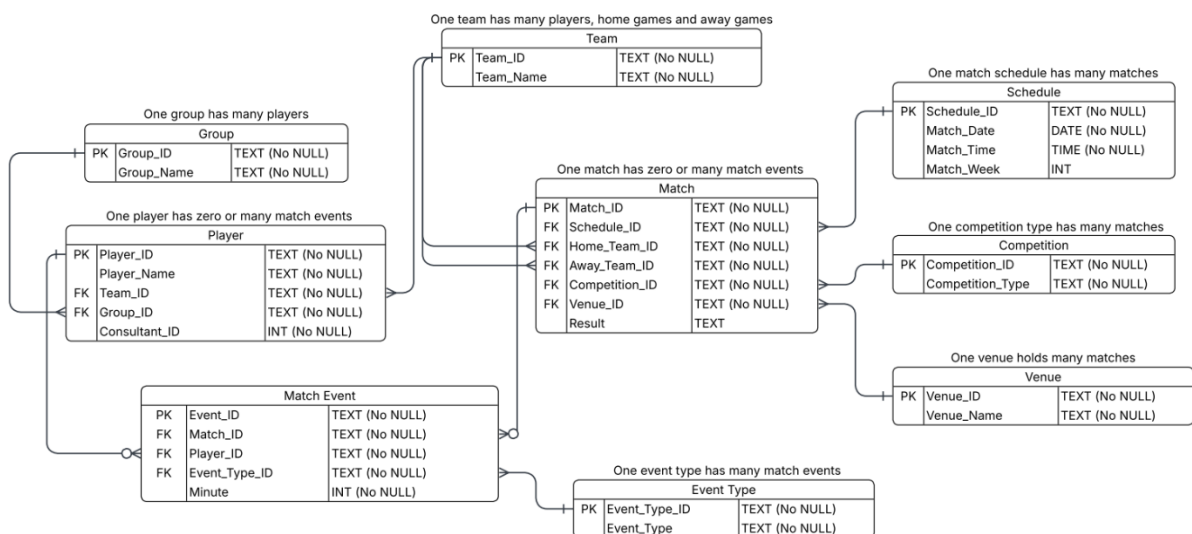
FOOTBALL LEAGUE DATABASE IMPLEMENTATION

- Clearly showed that **Home_Team_ID** and **Away_Team_ID** are foreign keys in the Match entity.

Original Entity-Relationship Diagram:



Updated Entity-Relationship Diagram:



These improvements levelled up my database design and prepared it perfectly for implementing it into PostgreSQL; further testing and analysis.

IMPLEMENTATION

(A more in-depth step by step tutorial on how to create the database is in the README file and the script can be found on page 10 of this document)

Database Setup

The database was implemented using **PostgreSQL**, following the structure defined in the entity-relationship design. The logical schema was translated into a relational model using SQL **CREATE TABLE** statements, with each entity represented as a table and attributes defined with appropriate data types.

Primary keys were established to uniquely identify each record, while foreign key constraints were used to enforce referential integrity between related tables. These relationships ensured consistency across the database, particularly between core entities such as teams, players, matches, schedules, and match events. Additional integrity constraints, including the use of **NOT NULL**, were applied to maintain required relationships and prevent incomplete data entries.

Data Population

The dataset was prepared externally on **Excel** and imported into the database using **CSV files**, ensuring that all tables were populated in accordance with their defined relationships.

Data Testing

Once implemented, the database was tested through a series of structured **SQL queries involving joins, aggregations, and filtering conditions**. These queries confirmed that the schema supports efficient data retrieval and accurately represents the relationships between entities.

Overview

Overall, the implementation successfully translated the conceptual and logical designs into a functional relational database capable of supporting complex queries and analysis.

DATA DICTIONARY**Group Table**

Column Name	Data Type	Description	Constraints	Relationships	Examples of Data
Group_ID	TEXT	Unique value given to each group name.	Primary Key (No NULL)	One group has many players.	G1, G2, G3
Group_Name	TEXT	The name of the groups.	No NULL	-	Cohort 4, Training Team

Player Table

Column Name	Data Type	Description	Constraints	Relationships	Examples of Data
Player_ID	TEXT	Unique value given to each individual player.	Primary Key (No NULL)	One player has zero or many match events.	P1, P2, P3
Player_Name	TEXT	Name of each individual player.	No NULL	-	Aedan Petty, Aliza Santos
Team_ID	TEXT	Unique value given to each team.	Foreign Key (No NULL)	One team has many players, home games and away games.	T1, T2, T3
Group_ID	TEXT	Unique value given to each group.	Foreign Key (No NULL)	One group has many players.	G1, G2, G3
Consultant_ID	INT	ID of the player's consultant.	No NULL	-	200, 201, 202

Match Event

Column Name	Data Type	Description	Constraints	Relationships	Examples of Data
Event_ID	TEXT	Unique value given to each individual event that happens in all the games.	Primary Key (No NULL)	-	E1, E2, E3
Match_ID	TEXT	Unique value given to each individual match played.	Foreign Key (No NULL)	One match has zero or many match events.	M1, M2, M3
Player_ID	TEXT	Unique value given to each individual player.	Foreign Key (No NULL)	One player has zero or many match events.	P1, P2, P3
Event_Type_ID	TEXT	Unique value given to each event type.	Foreign Key (No NULL)	One event type has many match events.	ET1, ET2, ET3
Minute	INT	The minute of the game that an event happened	>0 (No NULL)	-	5, 80, 10

Team

Column Name	Data Type	Description	Constraints	Relationships	Examples of Data
Team_ID	TEXT	Unique value given to each team.	Primary Key (No NULL)	One team has many players, home games and away games.	T1, T2, T3
Team_Name	TEXT	The name of each team.	No NULL	-	Data Masters, BI Gods

Match

Column Name	Data Type	Description	Constraints	Relationships	Examples of Data
Match_ID	TEXT	Unique value given to each individual match.	Primary Key (No NULL)	One match has zero or many match events.	M1, M2, M3
Schedule_ID	TEXT	Unique value given to each date, time and week a match is played at.	Foreign Key (No NULL)	One match schedule has many matches.	S1, S2, S3
Home_Team_ID	TEXT	Team_ID of the home team.	Foreign Key (No NULL)	One team has many players, home games and away games.	T1, T2, T3
Away_Team_ID	TEXT	Team_ID of the away team.	Foreign Key (No NULL)	One team has many players, home games and away games.	T1, T2, T3
Competition_ID	TEXT	Unique value given to the competition types.	Foreign Key (No NULL)	One competition type has many matches.	C1, C2
Venue_ID	TEXT	Unique value given to each venue a game can be played at.	Foreign Key (No NULL)	One venue holds many matches.	V1, V2, V3
Result	TEXT	The result of the match (Who wins or a draw).	Can only be one of 3 options.	-	Home Win, Away Win, Draw

Event Type

Column Name	Data Type	Description	Constraints	Relationships	Examples of Data
Event_Type_ID	TEXT	Unique value given to each type of event.	Primary Key (No NULL)	One event type has many match events.	ET1, ET2, ET3
Event_Type	TEXT	The name of the event.	No NULL	-	Goal, Yellow Card, Red Card

Schedule

Column Name	Data Type	Description	Constraints	Relationships	Examples of Data
Schedule_ID	TEXT	Unique value given to the date, time and week that a match is played on.	Primary Key (No NULL)	One match schedule has many matches.	S1, S2, S3
Match_Date	DATE	The date that the match is played on.	No NULL	-	2022-10-01, 2022-10-08
Match_Time	TIME	The time of the day that the match starts.	No NULL	-	16:00:00
Match_Week	INT	The match week the game is played on.	-	-	1, 2, 3

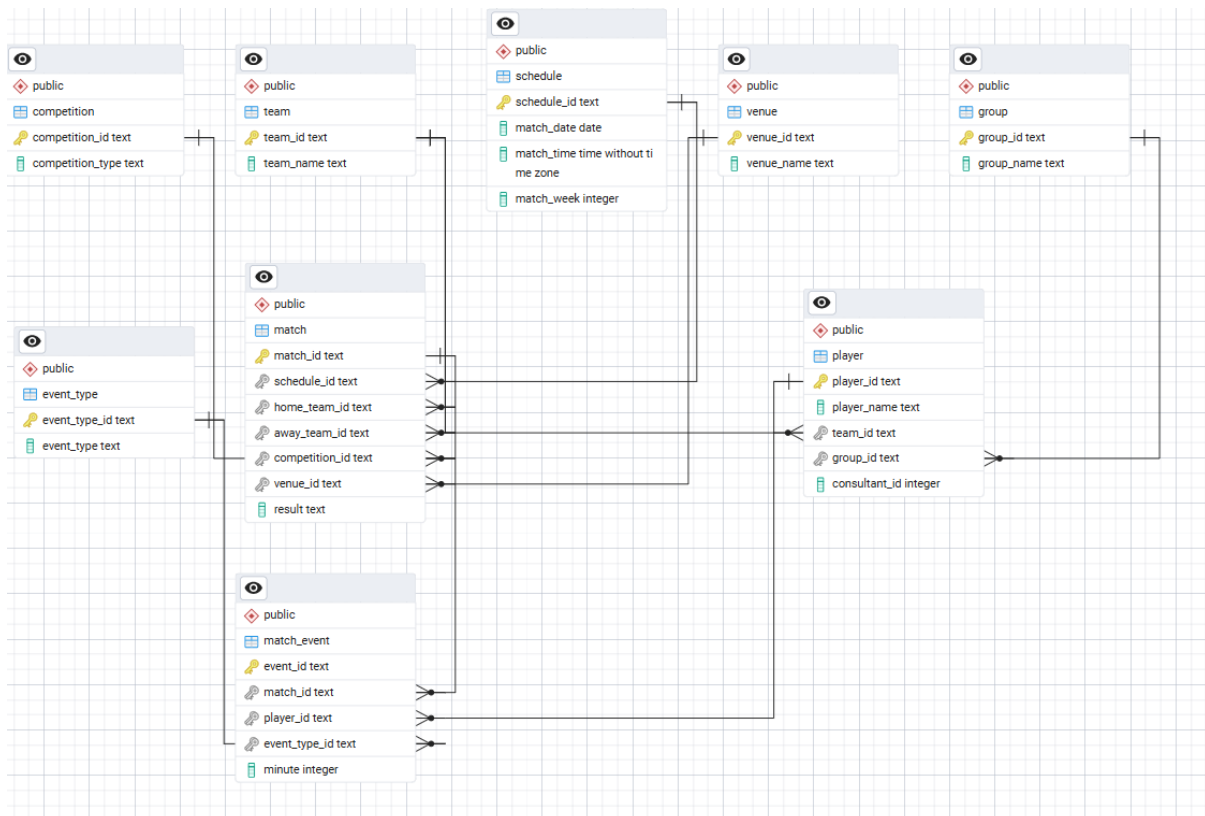
Competition

Column Name	Data Type	Description	Constraints	Relationships	Examples of Data
Competition_ID	TEXT	Unique value given to each competition type.	Primary Key (No NULL)	One competition type has many matches.	C1, C2
Competition_Type	TEXT	The types of competitions.	No NULL	-	Tournament, Friendly

Venue

Column Name	Data Type	Description	Constraints	Relationships	Examples of Data
Venue_ID	TEXT	Unique value given to each venue.	Primary Key (No NULL)	One venue holds many matches.	V1, V2, V3
Venue_Name	TEXT	The name of the venues.	No NULL	-	Wimbledon 1, Wimbledon 2, Wimbledon 3

POSTGRESQL ENTITY-RELATIONSHIP DIAGRAM



Differences

- **Manmade ERD is easier to follow.**
- **Manmade ERD has constraints shown.**
- **Manmade ERD has text** to explain relationships.
- **PostgreSQL ERD has all the relationships as one to zero or many.** This is because in data implementation, while foreign keys must reference a primary key, a primary key value may exist without being related to the foreign key. However, the manmade ERD represents the real-life constraints and understanding such as every player must be on a team, and every venue needs to be played on.

SCRIPT

Script used to create the tables:

-- Creating Team Table

```
CREATE TABLE Team (  
    team_id TEXT NOT NULL PRIMARY KEY,  
    team_name TEXT NOT NULL  
);
```

-- Creating Group Table

```
CREATE TABLE "group" (  
    group_id TEXT NOT NULL PRIMARY KEY,  
    group_name TEXT NOT NULL  
);
```

-- Creating Competition Table

```
CREATE TABLE Competition (  
    competition_id TEXT NOT NULL PRIMARY KEY,  
    competition_type TEXT NOT NULL  
);
```

-- Creating Venue Table

```
CREATE TABLE Venue (  
    venue_id TEXT NOT NULL PRIMARY KEY,  
    venue_name TEXT NOT NULL  
);
```

-- Creating Schedule Table

```
CREATE TABLE Schedule (
```

```
    schedule_id TEXT NOT NULL PRIMARY KEY,
```

```
    match_date DATE NOT NULL,
```

```
    match_time TIME NOT NULL,
```

```
    match_week INT
```

```
);
```

```
-- Creating Player Table
```

```
CREATE TABLE Player (
```

```
    player_id TEXT NOT NULL PRIMARY KEY,
```

```
    player_name TEXT NOT NULL,
```

```
    team_id TEXT NOT NULL REFERENCES Team(team_id),
```

```
    group_id TEXT NOT NULL REFERENCES "group"(group_id),
```

```
    consultant_id INT NOT NULL
```

```
);
```

```
-- Creating Match Table
```

```
CREATE TABLE Match (
```

```
    match_id TEXT NOT NULL PRIMARY KEY,
```

```
    schedule_id TEXT NOT NULL REFERENCES Schedule(schedule_id),
```

```
    home_team_id TEXT NOT NULL REFERENCES Team(team_id),
```

```
    away_team_id TEXT NOT NULL REFERENCES Team(team_id),
```

```
    competition_id TEXT NOT NULL REFERENCES Competition(competition_id),
```

```
    venue_id TEXT NOT NULL REFERENCES Venue(venue_id),
```

```
    result TEXT
```

```
);
```

```
-- Creating Event Type Table
```

```
CREATE TABLE Event_Type (  
    event_type_id TEXT NOT NULL PRIMARY KEY,  
    event_type TEXT NOT NULL  
);  
  
-- Creating Match Event Table  
  
CREATE TABLE Match_Event (  
    event_id TEXT NOT NULL PRIMARY KEY,  
    match_id TEXT NOT NULL REFERENCES Match(match_id),  
    player_id TEXT NOT NULL REFERENCES Player(player_id),  
    event_type_id TEXT NOT NULL REFERENCES Event_Type(event_type_id),  
    minute INT NOT NULL CHECK (minute>0)  
);  
  
/*Only columns that allow null values are the result column in the match table  
and the week column in the schedule table*/  
  
/*The minute column in the match event table must be greater than 0 as when  
storing football events, the minute can only be 1 and above*/
```

END OF SCRIPT

To find out how to insert the data from the csv files found in the folder along with this document into the tables made on PostgreSQL, go to the README file found in the same folder. This gives a step-by-step guide in how to insert the data.

TESTING

Test 1. Listing all students, who play for a particular department (i.e. Cohort 4 group).

To find all students who play for Cohort 4 we need to look at the original data of all the players and see who belongs to Cohort 4.

Consultant ID	Name	Team Name	Group Name
200	Aedan Petty	Data Masters	Cohort 4
201	Aliza Santos	Data Masters	Cohort 6
201	Kaylynn Vaughan	Data Masters	Training Team
202	Arjun Bauer	Data Masters	Cohort 7
202	Lillian Huber	Data Masters	Cohort 5
203	Lizeth Roberts	Data Masters	Training Team
203	Nathan McDowell	Data Masters	HR
204	Alvin Ali	Data Masters	Consultants
204	Jordin Christensen	Data Masters	HR
205	Saul Blevins	Data Masters	HR
205	Carina Meza	BI Gods	Cohort 5
206	Isabelle Campos	BI Gods	Bench
206	Kyleigh Phelps	BI Gods	Cohort 4
207	Angela Wong	BI Gods	HR
207	Kole Rojas	BI Gods	HR
208	Martha Potts	BI Gods	Cohort 7
208	Tomas Powell	BI Gods	Training Team
209	Paxton Clarke	BI Gods	Bench
209	Jamya Dodson	BI Gods	Consultants
210	Georgia Clements	BI Gods	Consultants
210	Edwin Crawford	Vis Wizards	Cohort 5
211	Malachi Osborn	Vis Wizards	Consultants
211	Zion Kent	Vis Wizards	Cohort 5
212	Anahi Reyes	Vis Wizards	Bench
212	Maddox Cabrera	Vis Wizards	Consultants
213	Brody Gutierrez	Vis Wizards	Cohort 7
213	Hayley Stevenson	Vis Wizards	Cohort 6
214	Kamora Sanchez	Vis Wizards	Consultants
214	Livia Holmes	Vis Wizards	Training Team
215	Tanner Jenkins	Vis Wizards	Consultants
215	Madelyn Meadows	Data Cleaners	Bench
216	Paola Wilkerson	Data Cleaners	Cohort 6
216	Jared Patton	Data Cleaners	Training Team
217	Pierre Washington	Data Cleaners	Consultants
217	Dominik Cochran	Data Cleaners	Cohort 7
218	Miya Skinner	Data Cleaners	Cohort 7
218	Mara Barnett	Data Cleaners	Bench
219	Cornelius Dodson	Data Cleaners	Cohort 5
219	Ashleigh Kaiser	Data Cleaners	Training Team
220	Weston Meza	Data Cleaners	Training Team

So, the expected result should be a table of the names Aedan Petty and Kyleigh Phelps.

```

1  --1. Listing all students, who play for a particular department (i.e. Cohort 4 group).
2  SELECT
3      p.player_name,
4      g.group_name
5  FROM Player p
6  JOIN "group" g ON p.group_id = g.group_id  --joining the group table to the player table using the primary/foreign key group_id
7  WHERE g.group_name = 'Cohort 4';
8

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	player_name text	group_name text
1	Aedan Petty	Cohort 4
2	Kyleigh Phelps	Cohort 4

Test 2. Listing all fixtures for a specific date (i.e. 29th of October 2022 and including team names and venues).

If we look at the original data, we were given we will find a table that shows us this information for matches played on 29/10/2022.

Date	Home team	Away team	Venue
29/10/2022 16:00	BI Gods	Data Masters	Wimbledon 1
29/10/2022 16:00	Data Cleaners	Vis Wizards	Wimbledon 3

Excluding the time, this is the sort of result we should expect.

```

9  /*2. Listing all fixtures for a specific date (i.e. 29th of October 2022 and including
10 team names and venues).*/
11 SELECT
12     m.match_id,
13     s.match_date,
14     t1.team_name AS home_team,
15     t2.team_name AS away_team,
16     v.venue_name
17 FROM Match m
18 --joining teams, schedule and venue on match by the primary/foreign key relating the 2 tables together.
19 JOIN Schedule s ON m.schedule_id = s.schedule_id
20 JOIN Team t1 ON m.home_team_id = t1.team_id
21 JOIN Team t2 ON m.away_team_id = t2.team_id
22 JOIN Venue v ON m.venue_id = v.venue_id
23 WHERE s.match_date = '2022-10-29';
24

```

Data Output Messages Notifications

Showing rows: 1 to 2

	match_id text	match_date date	home_team text	away_team text	venue_name text
1	M7	2022-10-29	BI Gods	Data Masters	Wimbledon 1
2	M8	2022-10-29	Data Cleaners	Vis Wizards	Wimbledon 3

Test 3. Listing all the players who have scored more than 2 goals.

If we look at the table of results from the original data we will find the goalscorers, looking at this, we should find the players who scored 2 or more goals.

Date - Time	Home Team * Player(s) scored ** Yellow card *** Red card		Away Team * Player(s) scored
01/10/2022 – 16:00	Data Masters * Alvin Ali (min 5) * Nathan McDowell (min 80)	2 – 0	BI Gods ** Isabelle Campos (min 10)
01/10/2022 – 16:00	Vis Wizards * Hayley Stevenson (min 11) ** Tanner Jenkins (min 30)	1 – 1	Data Cleaners * Dominik Cochran (min 10)
08/10/2022 – 16:00	Data Masters * Alvin Ali (min 2) * Alvin Ali (min 88) ** Aedan Petty (min 10) ** Jordin Christensen (min 70)	2 – 1	Vis Wizards * Hayley Stevenson (min 85)
08/10/2022 – 16:00	BI Gods * Jamya Dodson (min 20) * Tomas Powell (min 40)	2 – 1	Data Cleaners * Dominik Cochran (min 30) *** Paola Wilkerson (min 40)
22/10/2022 – 16:00	Data Masters ** Saul Blevins (min 35)	0 – 0	Data Cleaners ** Cornelius Dodson (min 88)
22/10/2022 – 16:00	BI Gods * Jamya Dodson (min 5)	1 – 2	Vis Wizards * Kamora Sanchez (min 2) * Hayley Stevenson (min 55)
29/10/2022 – 16:00	BI Gods * Jamya Dodson (min 1) * Martha Potts (min 82)	2 – 2	Data Masters * Alvin Ali (min 35) * Kaylynn Vaughan (min 77) ** Kaylynn Vaughan (min 60)
29/10/2022 – 16:00	Data Cleaners * Dominik Cochran (min 66) ** Dominik Cochran (min 10)	1 – 2	Vis Wizards * Maddox Cabrera (min 12) * Hayley Stevenson (min 68)
05/11/2022 – 16:00	Vis Wizards * Livia Holmes (min 20) * Livia Holmes (min 25) * Brody Gutierrez (min 85) ** Zion Kent (min 30)	3 – 2	Data Masters * Nathan McDowell (min 77) * Alvin Ali (min 89)
05/11/2022 – 16:00	Data Cleaners *** Paola Wilkerson (min 10)	0 – 1	BI Gods * Jamya Dodson (min 75)

12/11/2022 – 16:00	Data Cleaners * Dominik Cochran (min 25) * Mara Barnett (min 46)	2 – 0	Data Masters ** Aedan Petty (min 62)
12/11/2022 – 16:00	Vis Wizards * Brody Gutierrez (min 80) * Maddox Cabrera (min 90)	2 – 2	BI Gods * Jamya Dodson (min 29) * Paxton Clarke (min 87) ** Tomas Powell (min 42)

Knowing that * represents a goal we can see the players who scored more than 2 goals are: Alvin Ali, Dominik Cochran, Hayley Stevenson and Jamya Dodson.

```

24
25 --3. Listing all the players who have scored more than 2 goals
26 SELECT
27     p.player_name,
28     COUNT(*) AS goals_scored --counts goals scored
29 FROM Player p
30 /*joining match event on player and event type on match event
31 we can now see which player did what event*/
32 JOIN Match_Event me ON p.player_id = me.player_id
33 JOIN Event_Type et ON me.event_type_id = et.event_type_id
34 WHERE et.event_type = 'Goal' --selects the event type goals
35 GROUP BY p.player_name
36 HAVING COUNT(*) > 2;
37

```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	player_name text	goals_scored bigint
1	Alvin Ali	5
2	Dominik Cochran	4
3	Hayley Stevens...	4
4	Jamya Dodson	5

Test 4. Listing the number of cards (yellow and red) per team.

Using the same original data as Test 3 we can use the key that shows ** are yellow cards and *** are red cards. Counting these up gives us the total number of cards for each team. Data Masters got 5 cards, BI Gods got 2 cards, Vis Wizards got 2 cards and Data Cleaners got 4 cards.

```

38 --4. Listing the number of cards (yellow and red) per team.
39 SELECT
40     t.team_name,
41     COUNT(*) AS total_cards --counts total number of cards
42 FROM Team t
43 /*joining player on team, match event on player, event type on match event
44 allows us to see which players got cards and which team they play for*/
45 JOIN Player p ON t.team_id = p.team_id
46 JOIN Match_Event me ON p.player_id = me.player_id
47 JOIN Event_Type et ON me.event_type_id = et.event_type_id
48 WHERE et.event_type IN ('Yellow Card', 'Red Card') --selects all cards
49 GROUP BY t.team_name
50 ORDER BY total_cards DESC;
51

```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	team_name text	total_cards bigint
1	Data Masters	5
2	Data Cleaners	4
3	BI Gods	2
4	Vis Wizards	2

Test 5. Return the games that are going to be played (friendly matches).

Looking at the original data, we can see that there are 2 friendly games.

Friendly			
Date	Home team	Away team	Venue
12/11/2023 16:00	Data Cleaners	Data Masters	Wimbledon 1
12/11/2023 16:00	Vis Wizards	BI Gods	Wimbledon 3

Minus the venue, this should be how the result should look.

```

52  --5. Return the games that are going to be played (friendly matches).
53  SELECT
54      m.match_id,
55      s.match_date,
56      t1.team_name AS home_team,
57      t2.team_name AS away_team
58  FROM Match m
59  /*joining competition, schedule and team onto match allows us to
60  see which matches, when they are played and who is playing
61  for all the friendly games*/
62  JOIN Competition c ON m.competition_id = c.competition_id
63  JOIN Schedule s ON m.schedule_id = s.schedule_id
64  JOIN Team t1 ON m.home_team_id = t1.team_id
65  JOIN Team t2 ON m.away_team_id = t2.team_id
66  WHERE c.competition_type = 'Friendly'; --selects only the friendly games
67

```

Data Output Messages Notifications

Showing rows: 1 to 2 of 1

	match_id text	match_date date	home_team text	away_team text
1	M13	2023-11-12	Data Cleaners	BI Gods
2	M14	2023-11-12	Vis Wizards	Data Masters

Test 6. The table displays the team's name and the points earned during the tournament.

Using the original data looked at in tests 3 and 4 we can work out the points for each team by giving them 3 points for a win, 1 point for a draw and 0 points for a loss.

Doing this calculation, we find that Data Masters got 7 points, BI Gods got 8 points, Vis Wizards got 11 points and Data Cleaners got 4 points.

(The code is too long to present along side the result so the code will be looked at in more detail on the presentation)

	team_name text	total_points bigint
1	Vis Wizards	11
2	BI Gods	8
3	Data Masters	7
4	Data Cleaners	4

Test 7. For each team, present the distribution of goals scored and conceded in each half of the match.

Once again using that same original data we can calculate the goals scored and conceded by each team as we know that * represents goals.

Doing the calculations we get:

Data Masters scoring 3 in the first half and 5 in the second half. Conceding 4 in the first half and 4 in the second half.

BI Gods scoring 5 in the first half and 3 in the second half. Conceding 4 in the first half and 5 in the second half.

Vis Wizards scoring 5 in the first half and 6 in the second half. Conceding 4 in the first half and 5 in the second half.

Data Cleaners scoring 3 in the first half and 2 in the second. Conceding 4 in the first half and 2 in the second half.

(Once again, the code is too long to be shown alongside the results so I will go into a deeper look in the presentation)

	team_name text	goals_scored_1st_half bigint	goals_scored_2nd_half bigint	goals_conceded_1st_half bigint	goals_conceded_2nd_half bigint
1	BI Gods	5	3	4	5
2	Data Cleaners	3	2	4	2
3	Data Masters	3	5	4	4
4	Vis Wizards	5	6	4	5